



كلية علوم الحاسب والمعلومات
قسم تقنية المعلومات

King Saud University
College of Computer and Information Sciences
Department of Information Technology
IT462– Big Data Systems
Project Report

Prepared by:
Alanoud Alarifi - 442201803
Rema Alsharef - 442200419
Noura Alaskar - 442202216

Table of Content

1. Introduction	3
2. Data Selection	4
3. Data Preprocessing	5
3.1. Loading and Exploring the Data	5
3.2. Handling Missing Values	5
3.3. Encoding Categorical Features (Data Transformation)	6
3.4. Correlation Analysis	7
3.5. Feature Selection and Reduction	8
4. RDD Operations	10
4.1 RDD Transformations	10
4.2 RDD Actions	11
5. SQL Operations	13
5.1 SQL queries	13
6. Machine Learning Operations	17
6.1. Problem Description	17
6.2. Evaluation Metrics	18
6.3. Machine Learning Models	18
6.4. Hyperparameter Tuning Method	23
6.5. Results	30
7. Conclusion	31
8. colab code link	32

1. Introduction

Understanding the factors that influence student performance is crucial in today's educational landscape. This project analyzes a Student Performance Factors dataset to uncover insights that can enhance academic outcomes.

The dataset includes variables such as attendance, study habits, and tutoring sessions. By applying statistical analysis and data visualization, we aim to explore the relationships between these factors and student performance metrics like grades.

Our goal is to identify key influences on performance and highlight areas for intervention. Through this analysis, we hope to contribute to creating an environment where every student can succeed. and explore the stories behind the data and ways to improve educational experiences.

2. Data Selection

Data size :

- Number of Records (columns): 6,607
- Number of Features (rows): 20

URL: <https://www.kaggle.com/datasets/lainguyn123/student-performance-factors>

Content & format :

Attribute	Description
Hours_Studied	Number of hours spent studying per week.
Attendance	Percentage of classes attended.
Parental_Involvement	Level of parental involvement in the student's education (Low, Medium, High).
Access_to_Resources	Availability of educational resources (Low, Medium, High).
Extracurricular_Activities	Participation in extracurricular activities (Yes, No).
Sleep_Hours	Average number of hours of sleep per night.
Previous_Scores	Scores from previous exams.
Motivation_Level	Student's level of motivation (Low, Medium, High).

Internet_Access	Availability of internet access (Yes, No).
Tutoring_Sessions	Number of tutoring sessions attended per month.
Family_Income	Family income level (Low, Medium, High).
Teacher_Quality	Quality of the teachers (Low, Medium, High).
School_Type	Type of school attended (Public, Private).
Peer_Influence	Influence of peers on academic performance (Positive, Neutral, Negative).
Physical_Activity	Average number of hours of physical activity per week.
Learning_Disabilities	Presence of learning disabilities (Yes, No).
Parental_Education_Level	Highest education level of parents (High School, College, Postgraduate).
Distance_from_Home	Distance from home to school (Near, Moderate, Far).

Gender	Gender of the student (Male, Female).
Exam_Score	Final exam score.

3. Data Preprocessing

In this section, we'll discuss how we processed the data before moving on to the modeling.

3.1. Loading and Exploring the Data

First, we loaded the dataset "StudentPerformanceFactors.csv" into a Pandas DataFrame using `pd.read_csv()`. To get a quick overview of the data, we used `df.info()`, which gave us

using `pd.read_csv()`. To get a quick overview of the data, we used `df.info()`, which gave us some important details, like the number of non-null values, data types, and total columns. This step helped us see if there were any immediate data quality issues we needed to deal with.

```
Data columns (total 20 columns):
#   Column                               Non-Null Count  Dtype
---  -
0   Hours_Studied                        6607 non-null    int64
1   Attendance                          6607 non-null    int64
2   Parental_Involvement                 6607 non-null    object
3   Access_to_Resources                  6607 non-null    object
4   Extracurricular_Activities           6607 non-null    object
5   Sleep_Hours                         6607 non-null    int64
6   Previous_Scores                     6607 non-null    int64
7   Motivation_Level                    6607 non-null    object
8   Internet_Access                     6607 non-null    object
9   Tutoring_Sessions                   6607 non-null    int64
10  Family_Income                       6607 non-null    object
11  Teacher_Quality                     6529 non-null    object
12  School_Type                         6607 non-null    object
13  Peer_Influence                      6607 non-null    object
14  Physical_Activity                   6607 non-null    int64
15  Learning_Disabilities               6607 non-null    object
16  Parental_Education_Level            6517 non-null    object
17  Distance_from_Home                  6548 non-null    object
18  Gender                             6607 non-null    object
19  Exam_Score                         6607 non-null    int64
dtypes: int64(7), object(13)
```

- The number of rows in the dataset is 6607
- The column names and the data types
- The count of non-null values for each column (useful for identifying missing data).

Then, we checked for missing values using `df.isnull().sum()`. This was crucial for understanding how much data cleaning was required before moving forward, there were only 3 attributes with missing values which:

- Teacher_Quality (78)
- Distance_from_Home (67)
- Parental_Education_Level(90)

3.2. Handling Missing Values

Once we identified the missing values, we used `df.dropna()` to remove any rows with missing entries. After dropping the rows, we ran `df.isnull().sum()` again to ensure all missing values were gone.

```
[ ] df = df.dropna()

▼ Verifying the Removal of Missing Values

[ ] df.isnull().sum()
```

Hours_Studied	0	Family_Income	0
Attendance	0	Teacher_Quality	0
Parental_Involvement	0	School_Type	0
Access_to_Resources	0	Peer_Influence	0
Extracurricular_Activities	0	Physical_Activity	0
Sleep_Hours	0	Learning_Disabilities	0
Previous_Scores	0	Parental_Education_Level	0
Motivation_Level	0	Distance_from_Home	0
Internet_Access	0	Gender	0
Tutoring_Sessions	0	Exam_Score	0

We decided to drop rows instead of filling them because it seemed like the simplest way to clean the dataset at that point. However, we recognize that imputation techniques could have been used, especially if there were a lot of missing values or if they were concentrated in important features.

3.3. Encoding Categorical Features (Data Transformation)

Next, we identified several categorical columns in the dataset, such as `'Parental_Involvement'`, `'Access_to_Resources'`, and `'Gender'`. To better understand these columns, we printed the

```
# Print unique values for all categorical columns
categorical_columns = [
    'Parental_Involvement',
    'Access_to_Resources',
    'Motivation_Level',
    'Family_Income',
    'Teacher_Quality',
    'Distance_from_Home',
    'Gender',
    'Extracurricular_Activities',
    'Internet_Access',
    'Learning_Disabilities',
    'School_Type',
]
```

unique values for each categorical feature.

```
'Peer_Influence',  
'Parental_Education_Level'  
]  
  
unique_values = df[categorical_columns].apply(lambda col: col.unique())  
print(unique_values)
```

```
Parental_Involvement      [Low, Medium, High]  
Access_to_Resources       [High, Medium, Low]  
Motivation_Level         [Low, Medium, High]  
Family_Income            [Low, Medium, High]  
Teacher_Quality          [Medium, High, Low]  
Distance_from_Home       [Near, Moderate, Far]  
Gender                   [Male, Female]  
Extracurricular_Activities [No, Yes]  
Internet_Access          [Yes, No]  
Learning_Disabilities     [No, Yes]  
School_Type              [Public, Private]  
Peer_Influence           [Positive, Negative, Neutral]  
Parental_Education_Level [High School, College, Postgraduate]
```

After that, we used dictionaries to map these categories into numerical values. For example:

- 'Parental_Involvement' was mapped as {'Low': 1, 'Medium': 2, 'High': 3}.
- 'Gender' was mapped as {'Male': 0, 'Female': 1}.

We applied these mappings using the `.map()` function to convert categorical features to numerical ones. The goal here was to make the data usable in a machine learning model, as most machine learning models can only handle numerical inputs.

```
encodings = {  
    'Parental_Involvement': {'Low': 1, 'Medium': 2, 'High': 3},  
    'Access_to_Resources': {'Low': 1, 'Medium': 2, 'High': 3},  
    'Motivation_Level': {'Low': 1, 'Medium': 2, 'High': 3},  
    'Family_Income': {'Low': 1, 'Medium': 2, 'High': 3},  
    'Teacher_Quality': {'Low': 1, 'Medium': 2, 'High': 3},  
    'Distance_from_Home': {'Near': 1, 'Moderate': 2, 'Far': 3},  
    'Gender': {'Male': 0, 'Female': 1},  
    'Extracurricular_Activities': {'No': 0, 'Yes': 1},  
    'Internet_Access': {'No': 0, 'Yes': 1},  
    'Learning_Disabilities': {'No': 0, 'Yes': 1},  
    'School_Type': {'Public': 1, 'Private': 0},  
    'Peer_Influence': {'Positive': 2, 'Neutral': 1, 'Negative': 0},  
    'Parental_Education_Level': {'High School': 0, 'College': 1, 'Postgraduate': 2},  
}
```

```
for column, mapping in encodings.items():  
    df[column] = df[column].map(mapping)
```

3.4. Correlation Analysis

To better understand how our features were related to the target variable ('Exam_Score'), we computed the correlation matrix with `df.corr()`. We focused on the correlation values with 'Exam_Score' and sorted them to identify which features were most relevant. This helped us determine which features might have the most predictive power.

```
# Compute the correlation matrix  
corr_matrix = df.corr()  
  
# Display correlation with the target variable  
target_corr = corr_matrix['Exam_Score'].drop('Exam_Score')  
print(target_corr.sort_values(ascending=False))
```

```
Attendance          0  
Hours_Studied       0  
Previous_Scores     0  
Access_to_Resources 0  
Tutoring_Sessions   0  
Parental_Involvement 0  
Parental_Education_Level 0  
Peer_Influence      0  
Family_Income       0  
Motivation_Level    0
```

```
Teacher_Quality      0.  
Extracurricular_Activities 0.  
Internet_Access      0.  
Physical_Activity    0.  
Gender               0.  
School_Type         -0.  
Sleep_Hours         -0.  
Learning_Disabilities -0.  
Distance_from_Home  -0.
```


Interpretation of Correlation Values

- **Positive Correlation:** A positive value indicates that as the feature increases, the target variable (Exam_Score) also tends to increase. For example:
 - Attendance (0.580259) and Hours_Studied (0.445104) have strong positive correlations with Exam_Score, meaning higher attendance and more study hours are associated with higher exam scores.
- **Negative Correlation:** A negative value indicates that as the feature increases, the target variable tends to decrease. For example:
 - Distance_from_Home (-0.088083) has a slight negative correlation with Exam_Score, suggesting that a greater distance from home might be associated with a slightly lower exam score.
- **Low or Near Zero Correlation:** Features with values close to 0 have little to no linear relationship with the target variable. For example:
 - Gender (0.004932) and School_Type (-0.010868) have very low correlations with Exam_Score, indicating that these features have minimal linear impact on exam scores.

Data Reduction Implications

Based on the correlation values, features with very low or negative correlations (e.g., Gender, School_Type, Learning_Disabilities, Distance_from_Home) may be considered less important for predicting exam scores.

3.5. Feature Selection and Reduction

To further identify the most important features, we used a **RandomForestRegressor** to calculate feature importances. After training the model, we analyzed the feature importances and identified a few columns that had either very low or negative correlations with the target. Features such as 'Gender', 'School_Type', 'Learning_Disabilities', 'Extracurricular_Activities', and 'Internet_Access' had low importance, so we

```
from sklearn.ensemble import RandomForestRegressor
import numpy as np

# Define the target and features
X = df.drop(columns=['Exam_Score']) # Features (after data red
y = df['Exam_Score'] # Target variable

# Initialize the RandomForestRegressor
model = RandomForestRegressor(n_estimators=100, random_state=42)

# Fit the model
model.fit(X, y)
```

```
# Get feature importances
feature_importances = model.feature_importances_

# Create a DataFrame for better visualization
importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': feature_importances
}).sort_values(by='Importance', ascending=False)

# Display feature importances
print(importance_df)
```

decided to drop them.

The decision to drop these features was based on their weak correlation with '**Exam_Score**' and their minimal contribution to the model according to the feature importance scores. By removing them, we aimed to simplify the dataset, reduce the risk of overfitting, and make our model more efficient and easier to interpret.

	Feature	Importance
1	Attendance	0.384859
0	Hours_Studied	0.242514
6	Previous_Scores	0.082294
2	Parental_Involvement	0.038165
9	Tutoring_Sessions	0.034163
3	Access_to_Resources	0.033318
14	Physical_Activity	0.027151
5	Sleep_Hours	0.026478
16	Parental_Education_Level	0.018968
10	Family_Income	0.018341
13	Peer_Influence	0.016136
17	Distance_from_Home	0.014441
7	Motivation_Level	0.013897
11	Teacher_Quality	0.012340
15	Learning_Disabilities	0.009623
12	School_Type	0.007801
4	Extracurricular_Activities	0.007153
8	Internet_Access	0.006356
18	Gender	0.006002

Based on the results, it's clear that some features have significantly more importance than others for predicting the target variable.

As we can see starting with Attendance and hours studied which are the most important features going down to Learning_Disabilities, School_Type, Extracurricular_Activities, Internet_Access, and Gender having minimal impact, suggesting they could be removed without significantly affecting model performance, So we removed them:

```
[ ] from collections import defaultdict

# Columns to drop based on low or negative correlation with 'Exam_Score'
# and low importance from feature importance analysis
columns_to_drop = [
    'Gender', # Very weak positive correlation and low feature importance
    'School_Type', # Very weak negative correlation and low feature importance
    'Learning_Disabilities', # Negative correlation and low feature importance
    'Extracurricular_Activities', # Low feature importance based on Random Forest
    'Internet_Access' # Low feature importance based on Random Forest
```

```

]

# Drop the columns
df = df.drop(columns=columns_to_drop)

# Display the first few rows after reducing
df.head()

```

Summarizing of Steps in Preprocessing:

- We started by **loading and exploring the data** to understand its structure and check for missing values.
- We **handled missing values** by dropping rows that contained them, although we recognize that imputing them could have been a viable option as well.
- We then **encoded categorical variables** into numerical form to make them suitable for machine learning.
- After that, we performed a **correlation analysis** to see which features were most correlated with the target variable.
- Finally, we conducted **feature selection** based on correlation analysis and feature importance scores from a random forest model, dropping less useful features to improve efficiency.

Overall, our preprocessing steps focused on cleaning the data, making it suitable for machine learning, and reducing complexity to ensure better performance during modeling.

4. RDD Operations

4.1 RDD Transformations

Transformation 1: Map

We start by extracting relevant columns Hours_Studied and Exam_Score from the dataset.

```

11 scores = data.map(lambda line: line.split(",")).map(lambda col: (float(col[0]), float(col[1])))

```

This operation splits each line of data and then maps it to a tuple of study hours and exam scores.

Transformation 2: Filter

Transformation 2: Filter

We filter students who studied more than 5 hours.

```
1 | high_study_hours = scores.filter(lambda x: x[8] > 5)
```

This transformation filters rows where students studied for more than 5 hours.

Transformation 3: ReduceByKey

We calculate the average exam score of students who studied more than 5 hours

```
1 | total_scores = high_study_hours.mapValues(lambda x: (x, 1))\reduceByKey(lambda x, y: (x[0] + y[0], x[1] + y[1]))\  
average_high_study_scores = total_scores.mapValues(lambda x: x[0] / x[1])
```

This transformation sums the exam scores and counts the number of students, then calculates the average.

Transformation 4: Map

We create performance categories based on the Exam_Score column (High, Medium, Low).

```
1 | performance_levels = data.map(lambda row: row.split(",")[1].map(lambda index: ("Low" if float(row[index]) < 70 else ("Medium" if float(row[index]) < 85 else "High" if float(row[index]) < 90 else "Other" if float(row[index]) > 90)))
```

This operation maps exam scores to performance categories.

Transformation 5: GroupByKey

We group students based on their performance categories.

```
1 | grouped_performance = performance_levels.groupByKey()
```

4.2 RDD Actions

Action 1: Count

We count how many students studied more than 5 hours.

```

138 count_high_study = high_study_hours.count()
139
140 print(f"Number of students who studied more than 5 hours: {count_high_study}")
141
142 #> Number of students who studied more than 5 hours: 6000

```

Action 2: Collect

We collect the average exam scores of students who studied more than 5 hours.

```

143 collect_avg_high_study_scores = average_high_study_scores.collect()
144
145 print(f"Average exam scores of students who studied more than 5 hours: {collect_avg_high_study_scores}")
146
147 #> Average exam scores of students who studied more than 5 hours: [124.8, 66.287188697911, 128.8, 64.373400748280, 11-

```

Action 3: First

We retrieve the first entry in the performance levels RDD.

```

148 first_performance = performance_levels.first()
149
150 print(f"First performance entry: {first_performance}")
151
152 #> First performance entry: [07.R, 'Medium']

```

Action 4: Take

We get the top 10 exam scores.

```

153 top_exam_scores = scores.takeOrdered(10, key=lambda x: -x[1])
154
155 print(f"Top 10 exam scores: {top_exam_scores}")
156
157 #> Top 10 exam scores: [07.R, 180.0], [08.R, 180.0], [05.R, 180.0], [05.R, 180.0], [14.R, 180.0], [15.R, 180.0], [15.R, 180.0], [15.R, 180.0], [08.R, 180.0],

```

Action 5: SaveAsTextFile

We save the average scores to a text file.

```

158 average_high_study_scores.saveAsTextFile(f"{current_dir}/average_high_study_scores_new.txt")
159
160 print(f"File saved successfully.")
161
162 #> File saved successfully.

```

Summarizing of Steps in RDD:

project successfully applied five RDD transformation operations and five RDD action operations to analyze a student performance dataset. The operations provided insights into the impact of study hours on exam scores, categorized students by performance levels, and identified top performers. Overall, the analysis highlighted that study time is a critical factor in exam performance, while other factors such as parental involvement and access to resources could be explored further in future work.

5. SQL Operations

To support our analysis, we conducted several SQL operations on the dataset. in this section we will discuss the SQL queries and the results of each query.

5.1 SQL queries

Query 1: Calculate Average Exam Scores Grouped by Motivation Level: We used an SQL query to calculate average exam scores grouped by various categorical features, such as Motivation_Level and

```
Query 1: Calculate Average Exam Scores Grouped by Motivation Level

query1 = """
SELECT
    Motivation_Level,
    AVG(Exam_Score) AS Average_Exam_Score,
    COUNT(*) AS Number_of_Students
FROM
    StudentPerformanceFactors
GROUP BY
    Motivation_Level
ORDER BY
    Average_Exam_Score DESC;
"""

# Execute the query and fetch results
result1 = pd.read_sql_query(query1, conn)

print("1. Average Exam Scores Grouped by Motivation Level:")
print(result1)
```

Motivation_Level	Average_Exam_Score	Number_of_Students
3	67.743931	1277
2	67.338894	3237
1	66.764485	1864

Parental_Education_Level.

As the motivation level increases from 1 to 3, the average exam score also increases.

- Low Motivation (1): Average Score = 66.76
- Medium Motivation (2): Average Score = 67.34
- High Motivation (3): Average Score = 67.74

Insight: There is a slight positive correlation between motivation levels and exam scores, indicating that more motivated students tend to perform better academically.

Query 2: Identify Top 10 Students with Highest Exam Scores: we filter out students who scored above a certain threshold, allowing us to identify the characteristics of high-performing students.

Query 2: Identify Top 10 Students with Highest Exam Scores

```
query2 = """
SELECT
    Hours_Studied, Attendance, Parental_Involvement,
    Access_to_Resources, Sleep_Hours, Previous_Scores,
    Motivation_Level, Tutoring_Sessions, Family_Income,
    Teacher_Quality, Peer_Influence, Physical_Activity,
    Parental_Education_Level, Distance_from_Home, Exam_Score
FROM
    StudentPerformanceFactors
ORDER BY
    Exam_Score DESC
LIMIT 10;
"""

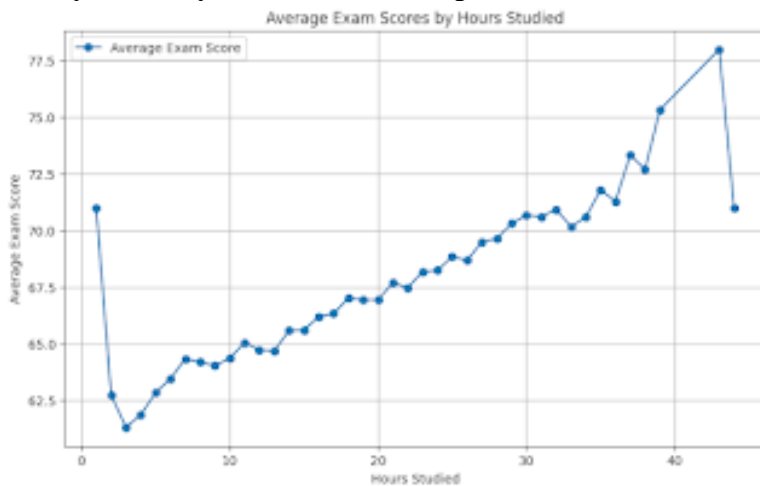
# Execute the query and fetch results
result2 = pd.read_sql_query(query2, conn)

print("\n2. Top 10 Students with Highest Exam Scores:")
print(result2)
```

this query prints a table of the Top 10 students and all their characteristics, the table is big but we interpreted the results of what they share:

- Hours Studied: The top scorers studied between 14 to 28 hours per week.
- Attendance: Attendance rates for these students are consistently high, ranging from 74% to 99%.
- Parental Involvement and Access to Resources: Parental Involvement: Students with high scores show varying levels of parental involvement. This indicates it is not always a critical predictor of top performance.
- Access to Resources: Access levels were generally moderate (mostly 2 or 3), showing that access to educational materials can aid in high performance.
- Sleep Hours: Sleep hours for these students range between 4 and 9 hours.
- Previous Scores: Most students with top current scores had high previous scores (73 to 98)
- Motivation Level: motivation levels vary from 1 to 3. Not all top students have the highest motivation level
- Family Income, Teacher Quality, and Peer Influence: These factors remain fairly consistent across the board, with moderate ratings (2 or 3), suggesting that, while they play a role, they may not be the most critical factors determining top exam scores.

Query 3: Analyze the Relationship Between Hours Studied and Exam Scores



Query 3: Analyze the Relationship Between Hours Studied and Exam Scores

```
query3 = """
SELECT
    Hours_Studied,
    AVG(Exam_Score) AS Average_Exam_Score,
    COUNT(*) AS Number_of_Students
FROM
    StudentPerformanceFactors
GROUP BY
    Hours_Studied
ORDER BY
    Hours_Studied;
"""

# Execute the query and fetch results
result3 = pd.read_sql_query(query3, conn)

print("\n3. Average Exam Scores by Hours Studied:")
print(result3)
```

we visualized the results of the query results to get more meaningful insights:

Low Study Hours (1-10 Hours):

- Average Exam Scores: Range between approximately 61 to 71.
- Sample Size: Varies from 1 to 91 students.

Moderate Study Hours (11-20 Hours):

- Average Exam Scores: Gradually increasing from ~64 to ~67.

- Sample Size: Significant increase, especially from 11 hours onwards.

High Study Hours (21-44 Hours):

- Average Exam Scores: Continuously rising from ~67 to ~78.
- Sample Size: Decreases as study hours increase, with very few students studying 43 and 44 hours.

Top Performers:

- Students studying between 37 to 44 hours show the highest average scores (72.71 to 78.00).
- However, the number of students in these categories is very low (1-7 students).
- Notable Anomaly: A student studying only 1 hour scored 71, which is higher than some students who studied more hours.

Query 4: The analysis of average exam scores by levels of parental involvement reveals a clear relationship between the degree of parental support and student performance. The results suggest a positive correlation between parental involvement and student performance, with higher levels of involvement associated with higher average exam scores. These findings underscore the critical role that parents play in supporting their children's education and indicate that increased parental engagement could be a key factor in improving student outcomes.

Query 4: Determine the Impact of Parental Involvement on Exam Scores

```
query4 = '''
SELECT
    Parental_Involvement,
    AVG(Exam_Score) AS Average_Exam_Score,
    COUNT(*) AS Number_of_Students
FROM
    StudentPerformanceFactors
GROUP BY
    Parental_Involvement
ORDER BY
    Average_Exam_Score DESC;
'''
```

```
# Execute the query and fetch results
result4 = pd.read_sql_query(query4, conn)

print("\n4. Average Exam Scores by Parental Involvement:")
print(result4)
```



```
4. Average Exam Scores by Parental Involvement:
  Parental_Involvement  Average_Exam_Score  Number_of_Students
0                    3          68.112288             1836
1                    2          67.113196             3251
2                    1          66.378776             1291
```

Query 5: Examine the Effect of Access to Resources on Exam Scores

Query 5: Examine the Effect of Access to Resources on Exam Scores



```
query5 = """
SELECT
    Access_to_Resources,
    AVG(Exam_Score) AS Average_Exam_Score,
    COUNT(*) AS Number_of_Students
FROM
    StudentPerformanceFactors
GROUP BY
    Access_to_Resources
ORDER BY
    Average_Exam_Score DESC;
"""

# Execute the query and fetch results
```

```
# Execute the query and fetch results
result5 = pd.read_sql_query(query5, conn)

print("\n5. Average Exam Scores by Access to Resources:")
print(result5)
```



5. Average Exam Scores by Access to Resources:			
	Access_to_Resources	Average_Exam_Score	Number_of_Students
0	3	68.103158	1900
1	2	67.156367	3204
2	1	66.223705	1274

The analysis of average exam scores by levels of access to resources provides important insights into the relationship between the availability of educational resources and student performance. The results demonstrate a positive correlation between access to educational resources and student exam performance. Students with higher levels of access consistently achieve better scores. This finding underscores the critical role that resource availability plays in student success and suggests that enhancing access to educational resources could lead to significant improvements in academic outcomes.

6. Machine Learning Operations

6.1. Problem Description

The primary task is to predict students' **exam scores** based on various features, such as previous academic performance, study hours, and other related attributes. This is a **regression problem** because the target variable (exam score) is continuous.

The goal is to build a machine learning model that can predict these scores as accurately as possible, which can help educators identify areas where students may need additional support or intervention. To prepare the dataset for modeling, the data was split into **training** and **testing sets**. This ensures that the models are trained on one portion of the data and tested on a separate unseen portion, providing a fair evaluation of their performance. The dataset was split as follows:

- **80% training data:** Used to train the models and allow them to learn patterns in the data.
- **20% testing data:** Used to evaluate the model's performance on unseen data.

The target variable (Exam_Score) was separated from the input features to ensure the models learn the relationships between the input features and the final exam scores.

Below is the code used to load the dataset, split the features and target, and partition the data into training and testing sets.

```
] # Load the dataset
import pandas as pd
from sklearn.model_selection import train_test_split

file_path = '/content/processed_dataset.csv'
student_data = pd.read_csv(file_path)

# Splitting features and target
X = student_data.drop(columns='Exam_Score') # Features
y = student_data['Exam_Score'] # Target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

6.2. Evaluation Metrics

The following metrics were consistently used to evaluate model performance, both before and after hyperparameter tuning

- **Mean Absolute Error (MAE):** Measures the average magnitude of the errors in the predictions, without considering their direction. A lower MAE means fewer errors.
- **Mean Squared Error (MSE):** Measures the squared difference between predicted and actual values. It gives higher weight to larger errors, which makes it useful for penalizing larger deviations.
- **R-squared (R^2):** Indicates how well the model explains the variance in the target variable. A higher R^2 value means the model fits the data better, with 1.0 being a perfect fit.

6.3. Machine Learning Models

In this section, we describe the machine learning models applied to the data, their purpose, and the results achieved. Screenshots of the relevant code for each model are included.

6.3.1. Linear Regression

Linear regression is one of the simplest and most interpretable models, assuming a linear relationship between the independent variables (features) and the dependent variable (exam score). It is fast to implement but might struggle to capture complex patterns or interactions between features. It serves as a

good baseline model to compare the performance of more complex algorithms.

Below is the code used to implement and evaluate Linear Regression:

```
[ ] # Linear Regression
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Initialize the Linear Regression model
lr_model = LinearRegression()

# Train the model
lr_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_lr = lr_model.predict(X_test)

# Evaluate the model
mae_lr = mean_absolute_error(y_test, y_pred_lr)
mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print(f"Linear Regression Results:")
print(f"MAE: {mae_lr}")
print(f"MSE: {mse_lr}")
print(f"R-squared: {r2_lr}")
```

Linear Regression Results:
MAE: 0.6347842984734274
MSE: 4.357166825416715
R-squared: 0.7196839862785757

1. Model Initialization:

- `LinearRegression()` initializes a linear regression model without additional parameters since it operates with default settings.

2. Model Training:

- The `.fit()` method trains the model using the training dataset (`X_train` and `y_train`).

3. Making Predictions:

- The `.predict()` method is used to predict exam scores for the test set (`X_test`).

4. Model Evaluation:

- The trained model is evaluated using metrics such as MAE, MSE, and R^2 (as previously described in the Evaluation Metrics section).

The Linear Regression model provides a reasonably good baseline, with an **R^2 of 0.720** indicating that approximately 72% of the variance in exam scores is explained by the input features. However the **MAE of 0.635** suggests a small error, but this may be further reduced using more sophisticated algorithms.

6.3.2. Decision Tree Regressor

Decision Trees are non-linear models that split the data into subsets based on certain feature values at each node. The final prediction is made by following the decision tree path from the root to a leaf node, which contains the prediction value.

Decision Trees can capture non-linear relationships and are easy to interpret. However, they are prone to overfitting, which is why hyperparameter tuning is essential.

Below is the code used to implement and evaluate the Decision Tree Regressor:

```
[ ] # Decision Tree Regressor
    from sklearn.tree import DecisionTreeRegressor

    # Initialize the Decision Tree model
    dt_model = DecisionTreeRegressor(random_state=42)

    # Train the model
    dt_model.fit(X_train, y_train)

    # Make predictions on the test set
    y_pred_dt = dt_model.predict(X_test)

    # Evaluate the model
    mae_dt = mean_absolute_error(y_test, y_pred_dt)
    mse_dt = mean_squared_error(y_test, y_pred_dt)
    r2_dt = r2_score(y_test, y_pred_dt)

    print(f"Decision Tree Regressor Results:")
    print(f"MAE: {mae_dt}")
    print(f"MSE: {mse_dt}")
    print(f"R-squared: {r2_dt}")
```

Decision Tree Regressor Results:
MAE: 1.8981191222570533
MSE: 16.178683385579937
R-squared: -0.04114454129354872

1. Model Initialization:

- ♦ `DecisionTreeRegressor(random_state=42)` initializes the Decision Tree Regressor, with the `random_state` parameter set to ensure consistent results across runs.

2. Model Training:

- ♦ The `.fit()` method trains the model on the training dataset (`X_train` and `y_train`).

3. Making Predictions:

- ♦ The `.predict()` method generates predictions for the test dataset (`X_test`).

4. Model Evaluation:

- ♦ The trained model is evaluated using MAE, MSE, and R^2 metrics (as previously described in the Evaluation Metrics section).

The Decision Tree Regressor performed poorly, with an **R^2 of -0.041**, indicating that the model failed to explain any variance in the target variable and may have overfit the training data. Additionally, the **MAE of 1.898** and **MSE of 16.179** suggest significant prediction errors. These results emphasize the need for hyperparameter tuning to constrain the model's complexity and improve performance.

6.3.3. Random Forest Regressor

Random Forest is an ensemble learning technique that builds multiple decision trees during training. It

averages the predictions from all trees to reduce variance and overfitting.

Random Forest is generally more robust than a single decision tree and provides a better balance between bias and variance. Additionally, it provides insight into feature importance.

Below is the code used to implement and evaluate the Random Forest Regressor:

```
# Random Forest Regressor
from sklearn.ensemble import RandomForestRegressor

# Initialize the Random Forest model
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)

# Train the model
rf_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_rf = rf_model.predict(X_test)

# Evaluate the model
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print("Random Forest Regressor Results:")
print(f"MAE: {mae_rf}")
print(f"MSE: {mse_rf}")
print(f"R-squared: {r2_rf}")

# Optional: Feature Importance
feature_importances = pd.DataFrame(rf_model.feature_importances_,
                                   index=X_train.columns,
                                   columns=['Importance']).sort_values('Importance', ascending=False)

print("Feature Importances:\n", feature_importances)
```

Random Forest Regressor Results:
MAE: 1.1324839561138536
MSE: 5.838492495297886
R-squared: 0.6242762435881772
Feature Importances:

	Importance
Attendance	0.382685
Hours_Studied	0.249292
Previous_Scores	0.092138
Parental_Involvement	0.037255
Tutoring_Sessions	0.036739
Access_to_Resources	0.034966
Physical_Activity	0.030256
Sleep_Hours	0.028138
Family_Income	0.023551
Parental_Education_Level	0.018804
Peer_Influence	0.017864
Distance_from_Home	0.016845
Motivation_Level	0.015282
Teacher_Quality	0.015178

1. Model Initialization:

- `RandomForestRegressor(n_estimators=100, random_state=42)` initializes the Random Forest model with 100 decision trees (`n_estimators`), ensuring consistency with the `random_state` parameter.

2. Model Training:

- The `.fit()` method trains the model using the training dataset (`X_train` and `y_train`).

3. Making Predictions:

- The `.predict()` method is used to predict exam scores for the test set (`X_test`).

4. Feature Importance:

- The `feature_importances_` attribute ranks the importance of each feature in making predictions, which can be visualized or analyzed for insights.

5. Model Evaluation:

- The trained model is evaluated using metrics such as MAE, MSE, and R^2 (as previously described in the Evaluation Metrics section).

The Random Forest Regressor achieved an **R^2 of 0.624**, indicating that approximately 62.4% of the variance in exam scores is explained by the model. The **MAE of 1.132** and **MSE of 5.838** suggest that the model performs better than the Decision Tree but has room for improvement. The feature importance analysis revealed that **Attendance** and **Hours_Studied** were the most influential features in predicting exam scores.

6.3.4. Gradient Boosting Regressor

Gradient Boosting builds models sequentially, each new model trying to correct the errors of its predecessor. This results in a strong model capable of handling complex patterns.

Gradient Boosting is powerful and often yields excellent results in regression tasks, though it may require hyperparameter tuning to avoid overfitting.

Below is the code used to implement and evaluate the Gradient Boosting Regressor:

```
[ ] # Gradient Boosting Regressor
    from sklearn.ensemble import GradientBoostingRegressor

    # Initialize the Gradient Boosting model
    gb_model = GradientBoostingRegressor(n_estimators=100, random_state=42)

    # Train the model
    gb_model.fit(X_train, y_train)

    # Make predictions on the test set
    y_pred_gb = gb_model.predict(X_test)

    # Evaluate the model
    mae_gb = mean_absolute_error(y_test, y_pred_gb)
    mse_gb = mean_squared_error(y_test, y_pred_gb)
    r2_gb = r2_score(y_test, y_pred_gb)

    print("Gradient Boosting Regressor Results:")
    print(f"MAE: {mae_gb}")
    print(f"MSE: {mse_gb}")
    print(f"R-squared: {r2_gb}")

↵ Gradient Boosting Regressor Results:
MAE: 0.8745443961879753
MSE: 4.7487721298328935
R-squared: 0.6944829336015571
```

1. Model Initialization:

- ♦ `GradientBoostingRegressor(n_estimators=100, random_state=42)` initializes the Gradient Boosting model with 100 boosting stages (`n_estimators`), ensuring reproducibility with the `random_state` parameter.

2. Model Training:

- ♦ The `.fit()` method trains the model using the training dataset (`X_train` and `y_train`).

3. Making Predictions:

- ♦ The `.predict()` method is used to predict exam scores for the test set (`X_test`).

4. Model Evaluation:

- ♦ The trained model is evaluated using metrics such as MAE, MSE, and R^2 (as previously described in the Evaluation Metrics section).

The Gradient Boosting Regressor performed well with an **R^2 of 0.694** indicating that approximately

The Gradient Boosting Regressor performed well, with an **R² of 0.694**, indicating that approximately 69.4% of the variance in exam scores is explained by the model. Additionally, the **MAE of 0.875** and **MSE of 4.749** suggest smaller prediction errors compared to the Decision Tree and Random Forest models. These results demonstrate the strength of Gradient Boosting in handling complex data patterns, though further optimization may improve performance.

6.4. Hyperparameter Tuning Method

To further improve the models' performance, hyperparameter tuning was performed using Randomized Search Cross-Validation. This method allows us to explore a wide range of hyperparameter values for each model, selecting the best combination based on cross-validation performance. Here's how it was applied for each model:

6.4.1 Gradient Boosting Regressor:

Gradient Boosting is a powerful machine learning algorithm that builds models sequentially, with each new model correcting the errors of its predecessor. To improve the performance of the Gradient Boosting Regressor, hyperparameter tuning was performed using **Randomized Search Cross-Validation**. This method efficiently explores a predefined grid of hyperparameters to identify the best configuration for the model.

The following hyperparameters were tuned:

- **Number of Boosting Stages (n_estimators):** The number of boosting iterations.
- **Learning Rate (learning_rate):** Controls the contribution of each tree to the final prediction.
- **Maximum Depth (max_depth):** Limits the depth of the individual trees to prevent overfitting.
- **Subsample Size (subsample):** The fraction of samples used for fitting each tree.
- **Minimum Samples Split and Leaf (min_samples_split and min_samples_leaf):** Parameters to control tree structure and avoid overly complex splits.

Below is the code used to perform hyperparameter tuning and evaluate the tuned Gradient Boosting Regressor:

```
from sklearn.ensemble import GradientBoostingRegressor

# Define the hyperparameter grid for Gradient Boosting
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.05, 0.1, 0.15],
    'max_depth': [3, 4, 5, 6],
    'min_samples_split': [10, 20, 30],
    'min_samples_leaf': [5, 10, 20],
    'subsample': [0.5, 0.8]
}

# Initialize the Gradient Boosting model
model = GradientBoostingRegressor(random_state=42)

# Randomized search
random_search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_grid,
    n_iter=10,  # Number of random configurations to try
    cv=5,      # 5-fold cross-validation
    verbose=1,
    random_state=42,
    n_jobs=-1  # Use all available cores
)

# Fit the random search model
random_search.fit(X_train, y_train)

# Best hyperparameters
print(f"Best Gradient Boosting parameters: {random_search.best_params_}")

# Evaluate the best model on the test set
gb_model = random_search.best_estimator_
y_pred_gb_tuned = gb_model.predict(X_test)
```

```

y_gb_tuned = mean_absolute_error(y_test, y_gb_tuned)
y_gb_tuned = mean_squared_error(y_test, y_gb_tuned)
y_gb_tuned = ci_gb_tuned_gb, y_gb_tuned_gb

[Out]: Tuned Gradient Boosting MSE: [y_gb_tuned]
[Out]: Tuned Gradient Boosting MAE: [y_gb_tuned]
[Out]: Tuned Gradient Boosting R-squared: [ci_gb_tuned]

Fitting 5 folds for each of 36 candidates, resulting in 180 fits
[Out]: Gradient Boosting parameters: {'learning_rate': 0.05, 'max_depth': 3, 'max_samples_split': 5, 'max_samples_leaf': 4, 'min_samples_split': 5, 'min_samples_leaf': 4}
[Out]: Gradient Boosting MSE: 0.7820000000000001
[Out]: Gradient Boosting MAE: 0.6220000000000001
[Out]: Gradient Boosting R-squared: 0.7020000000000001

```

1. Hyperparameter Grid Definition:

- A range of values for `n_estimators`, `learning_rate`, `max_depth`, and other tree parameters was specified to explore possible configurations.

2. Randomized Search Cross-Validation:

- The `RandomizedSearchCV` function was used to evaluate combinations of hyperparameters using 5-fold cross-validation.

3. Model Training:

- The best hyperparameters were identified, and the model was trained using these optimal values.

4. Model Evaluation:

- The trained model is evaluated using metrics such as MAE, MSE, and R^2 (as previously described in the Evaluation Metrics section).

The best hyperparameters identified for the Gradient Boosting Regressor were:

- **Subsample:** 0.8
- **Number of Estimators (`n_estimators`):** 300
- **Minimum Samples Split (`min_samples_split`):** 5
- **Minimum Samples Leaf (`min_samples_leaf`):** 4
- **Maximum Depth (`max_depth`):** 3
- **Learning Rate (`learning_rate`):** 0.05

The **R^2 of 0.702** indicates that approximately 70.2% of the variance in exam scores is explained by the model. The **MAE of 0.782** and **MSE of 4.632** highlight improved accuracy and reduced errors compared to the untuned model. These results demonstrate that tuning the boosting stages, learning rate, and subsample size effectively enhances model performance.

6.4.2 Decision Tree Regressor:

Hyperparameters such as max depth and minimum samples split were optimized to reduce overfitting and improve model accuracy.

Decision Trees are flexible models capable of capturing non-linear relationships, but they are prone to overfitting without proper constraints. To improve the performance of the Decision Tree Regressor and reduce overfitting, hyperparameter tuning was performed using **Randomized Search Cross-Validation**. This method efficiently identifies the best configuration of hyperparameters by exploring a specified grid.

The following hyperparameters were tuned:

- **Maximum Depth (max_depth):** Limits the depth of the tree to avoid overfitting while maintaining sufficient complexity.
- **Minimum Samples Split (min_samples_split):** The minimum number of samples required to split an internal node.
- **Minimum Samples Leaf (min_samples_leaf):** The minimum number of samples required at a leaf node to prevent small splits.

Below is the code used to perform hyperparameter tuning and evaluate the tuned Decision Tree Regressor:

```
from sklearn.tree import DecisionTreeRegressor

# Define the hyperparameter grid for Decision Tree
dt_param_grid = {
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Initialize the Decision Tree model
dt_model = DecisionTreeRegressor(random_state=42)

# Randomized search
dt_random_search = RandomizedSearchCV(estimator=dt_model,
                                       param_distributions=dt_param_grid,
                                       n_iter=50, # Number of random configurations
                                       cv=5, # 5-fold cross-validation
                                       verbose=2,
                                       random_state=42,
                                       n_jobs=-1) # use all available cores

# Fit the random search model
dt_random_search.fit(X_train, y_train)

# Best hyperparameters
print(f"Best Decision Tree parameters: {dt_random_search.best_params_}")

# Evaluate the best model on the test set
best_dt_model = dt_random_search.best_estimator_
y_pred_dt_tuned = best_dt_model.predict(X_test)

# Metrics for tuned Decision Tree
mae_dt_tuned = mean_absolute_error(y_test, y_pred_dt_tuned)
mse_dt_tuned = mean_squared_error(y_test, y_pred_dt_tuned)
r2_dt_tuned = r2_score(y_test, y_pred_dt_tuned)

print(f"Tuned Decision Tree MAE: {mae_dt_tuned}")
print(f"Tuned Decision Tree MSE: {mse_dt_tuned}")
print(f"Tuned Decision Tree R-squared: {r2_dt_tuned}")

/usr/local/lib/python3.10/dist-packages/sklearn/model_selection/_search.py:329:
Warning:
Fitting 3 folds for each of 36 candidates, totalling 108 fits
Best Decision Tree parameters: {'min_samples_split': 10, 'min_samples_leaf': 4,
Tuned Decision Tree MAE: 1.5085488537518357
Tuned Decision Tree MSE: 7.476388785888381
Tuned Decision Tree R-squared: 0.5187494442548488
```

1. Hyperparameter Grid Definition:

- A range of values for max_depth, min_samples_split, and min_samples_leaf was specified to control tree growth and complexity.

2. Randomized Search Cross-Validation:

- The RandomizedSearchCV function was used to evaluate 36 possible combinations of hyperparameters using 5-fold cross-validation.

3. Model Training:

- The best hyperparameters were identified, and the model was trained using these optimal values.

4. Model Evaluation:

- The trained model is evaluated using metrics such as MAE, MSE, and R^2 (as previously described in the Evaluation Metrics section).

The best hyperparameters identified for the Decision Tree Regressor were:

- **Maximum Depth (max_depth):** 10
- **Minimum Samples Split (min_samples_split):** 10
- **Minimum Samples Leaf (min_samples_leaf):** 4

The **R² of 0.519** indicates that the tuned model explains 51.9% of the variance in exam scores. Although the performance improved compared to the untuned model, the Decision Tree Regressor still underperformed relative to ensemble methods such as Random Forest and Gradient Boosting. This highlights the limitations of single-tree models and the importance of ensemble techniques for better accuracy.

6.4.3 Random Forest Regressor:

Random Forest is an ensemble learning technique that builds multiple decision trees during training and averages their predictions to reduce variance and overfitting. To enhance the performance of the Random Forest Regressor, hyperparameter tuning was performed using **Randomized Search Cross-Validation**. This method allows for an efficient exploration of a predefined range of hyperparameter combinations.

The following hyperparameters were tuned:

- **Number of Estimators (n_estimators):** The number of trees in the forest.
- **Maximum Depth (max_depth):** Limits the depth of each tree to prevent overfitting.
- **Minimum Samples Split (min_samples_split):** The minimum number of samples required to split an internal node.
- **Minimum Samples Leaf (min_samples_leaf):** The minimum number of samples required to be at a leaf node.
- **Bootstrap (bootstrap):** Whether to use bootstrap samples for building trees.

Below is the code used to implement and evaluate the tuned Random Forest Regressor:

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestRegressor

# Define the hyperparameter grid for Random Forest
param_grid = {
    'n_estimators': [100, 200, 300, 400, 500],
    'max_depth': [None, 10, 20, 30, 40, 50],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False]
}

# Initialize the Random Forest model
model = RandomForestRegressor(random_state=42)

# Randomized search
random_search = RandomizedSearchCV(estimator=model,
                                    param_distributions=param_grid,
                                    n_iter=10, # Number of random configurations to try
                                    cv=5, # 5-fold cross-validation
                                    verbose=0,
                                    random_state=42,
                                    n_jobs=-1) # Use all available cores

# Fit the random search model
random_search.fit(X_train, y_train)

# Best hyperparameters
best_params = random_search.best_params_

# Evaluate the best model on the test set
best_model = random_search.best_estimator_
pred_rf_tuned = best_model.predict(X_test)

# Metrics for tuned Random Forest
mse_rf_tuned = mean_squared_error(y_test, y_pred_rf_tuned)
r2_rf_tuned = r2_score(y_test, y_pred_rf_tuned)
f1_rf_tuned = f1_score(y_test, y_pred_rf_tuned)

print(f"Tuned Random Forest MSE: {mse_rf_tuned}")
print(f"Tuned Random Forest R2: {r2_rf_tuned}")
print(f"Tuned Random Forest F1-score: {f1_rf_tuned}")

# Print 5 folds for each of 50 candidates, totalling 250 fits
print("Random Forest parameters: {'n_estimators': 300, 'min_samples_split': 2, 'min_samples_leaf': 4, 'max_depth': 30, 'bootstrap': True}")
print("Random Forest MSE: 1.815088044210888")
print("Random Forest R2: 0.323436257304884")
print("Random Forest F1-score: 0.573229342787084")
```

1. Hyperparameter Grid Definition:

- A grid of values for n_estimators, max_depth, min_samples_split, min_samples_leaf, and bootstrap was defined to explore potential configurations.

2. Randomized Search Cross-Validation:

```
# XGBoost Regressor
from xgboost import XGBRegressor
from sklearn.model_selection import RandomizedSearchCV

# Define the hyperparameter grid for XGBoost
xgb_params_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 4, 5, 6],
    'subsample': [0.7, 0.8, 1.0],
    'colsample_bytree': [0.7, 0.8, 1.0]
}

# Train the XGBoost model
xgb_model = XGBRegressor(random_state=42)

# Randomized search
xgb_random_search = RandomizedSearchCV(estimator=xgb_model,
```

```

n_estimators,
    colsample_bytree,
    verbose=0,
    random_state=42,
    n_jobs=-1)

# Fit the random search model
xgb_random_search.fit(X_train, y_train)

# Best hyperparameters
print(f"Best XGBoost parameters: {xgb_random_search.best_params_}")

# Evaluate the best model on the test set
best_xgb_model = xgb_random_search.best_estimator_
y_pred_xgb_tuned = best_xgb_model.predict(X_test)

# Metrics for tuned XGBoost
mae_xgb_tuned = mean_absolute_error(y_test, y_pred_xgb_tuned)
mse_xgb_tuned = mean_squared_error(y_test, y_pred_xgb_tuned)
r2_xgb_tuned = r2_score(y_test, y_pred_xgb_tuned)

print(f"Tuned XGBoost MAE: {mae_xgb_tuned}")
print(f"Tuned XGBoost MSE: {mse_xgb_tuned}")
print(f"Tuned XGBoost R-squared: {r2_xgb_tuned}")

# Fitting 5 folds for each of 50 candidates, totalling 250 fits
Best XGBoost parameters: {'colsample_bytree': 0.7, 'n_estimators': 300, 'max_depth': 3, 'learning_rate': 0.05, 'colsample_bytree': 0.7}
Tuned XGBoost MAE: 0.760738177115833
Tuned XGBoost MSE: 4.5705461615460405
Tuned XGBoost R-squared: 0.705281748846875

```

1. Hyperparameter Grid Definition:

- A range of values for `n_estimators`, `learning_rate`, `max_depth`, `subsample`, and `colsample_bytree` was specified to explore different configurations.

2. Randomized Search Cross-Validation:

- The `RandomizedSearchCV` function was used to evaluate 50 combinations of hyperparameters using 5-fold cross-validation.

3. Model Training:

- The best hyperparameters were identified, and the model was trained using these optimal values.

4. Model Evaluation:

- The trained model is evaluated using metrics such as MAE, MSE, and R^2 (as previously described in the Evaluation Metrics section).

The best hyperparameters identified for the XGBoost Regressor were:

- **Subsample:** 0.7
- **Number of Estimators (`n_estimators`):** 300
- **Maximum Depth (`max_depth`):** 3
- **Learning Rate (`learning_rate`):** 0.05
- **Column Subsampling (`colsample_bytree`):** 0.7

The **R^2 of 0.705** indicates that approximately 70.5% of the variance in exam scores is explained by the model. The **MAE of 0.761** and **MSE of 4.580** highlight the effectiveness of XGBoost in reducing prediction errors compared to other models. These results demonstrate that tuning parameters such as boosting rounds, learning rate, and tree depth significantly improves the model's performance, making XGBoost the best-performing model in this project.

6.5. Results

6.5.1. Results without Hyperparameter Tuning

Model	MAE	MSE	R-squared
Linear Regression	0.635	4.36	0.720
Decision Tree	1.898	16.18	-0.041
Random Forest	1.132	5.84	0.624
Gradient Boosting	0.875	4.75	0.694

6.5.2. Results with Hyperparameter Tuning

Model	MAE	MSE	R-squared
Decision Tree	1.589	7.48	0.519
Random Forest	1.076	5.32	0.657
Gradient Boosting	0.782	4.63	0.702
XGBoost	0.761	4.58	0.705

From the results, it is clear that ensemble methods like Random Forest, Gradient Boosting, and XGBoost perform significantly better than simpler models like Linear Regression and Decision Tree

Regressors. Tuning further enhances their performance, particularly with XGBoost, which proved to be the most accurate in predicting students' exam scores.

XGBoost achieved an R^2 of **0.705**, explaining approximately 70.5% of the variance in exam scores, and demonstrated lower error rates (MAE: **0.761**, MSE: **4.580**) compared to other models. Its ability to capture complex relationships between features and target variables makes it a robust and reliable choice for this task, offering the highest predictive accuracy among the models evaluated.

7. Conclusion

In this project, we applied various techniques learned throughout the course to analyze a dataset of student performance factors, with the goal of predicting exam scores and gaining insights into the relationships between key variables like attendance, study habits, and other educational influences. The dataset was thoroughly preprocessed, which involved handling missing values, encoding categorical variables, and reducing the feature set to retain only the most relevant attributes.

We performed several **RDD transformation and action operations** to explore the dataset, such as mapping study hours to exam scores and categorizing students based on performance levels. These operations highlighted that study time is a critical factor in exam performance, with significant variations observed across different performance categories.

The **SQL operations** allowed us to analyze the data from multiple angles. We were able to group students by motivation levels and examine how factors such as parental involvement and access to educational resources impacted exam scores. The SQL queries also revealed insights into the top-performing students, showing how variables like study hours and attendance correlate with academic success.

For the machine learning phase, we tackled the problem of predicting exam scores by implementing multiple regression models, including **Linear Regression, Decision Tree, Random Forest, Gradient Boosting, and XGBoost**. After tuning the hyperparameters, **XGBoost** emerged as the best-performing model, providing the highest predictive accuracy and demonstrating its robustness in capturing complex relationships between the features and the target variable.

Overall, this project successfully applied big data techniques to gain actionable insights into student performance. It highlighted the importance of specific educational factors and demonstrated the power of machine learning in predicting academic outcomes. The results suggest that educators can use these insights to better tailor interventions for students and improve overall academic performance.

8. colab code link

- [BigDataProject.ipynb](#)